

# ACSC qualifiers 2023 Crypto - SusCipher

deuterium — 2023-03-01

Canonical HTML: <https://deut-erium.github.io/WriteUps/2023/ACSC/crypto/SusCipher/2023-03-01-ACSC-SusCipher.html>

## Abstract

CTF writeup: ACSC 2023's SusCipher gets cross-examined by Z3: 48 single-bit plaintexts give a model enough evidence to recover the three-round block cipher's key.

## Challenge Description

SusCipher 400 pts (6 solves)

authored by rbtrees

I made SusCipher, which is a vulnerable block cipher so everyone can break it!

Please, try it and find a key. nc suscipher.chal.ctf.acsc.asia 13579 nc suscipher-2.chal.ctf.acsc.asia 13579 (Backup) Hint: Differential cryptanalysis is useful.

SusCipher.tar.gz

## Source files

[task.py](#)

## Source Analysis

```
#!/usr/bin/env python3
import hashlib
import os
import signal

class SusCipher:
    S = [
        43, 8, 57, 53, 48, 39, 15, 61,
        7, 44, 33, 9, 19, 41, 3, 14,
        42, 51, 6, 2, 49, 28, 55, 31,
        0, 4, 30, 1, 59, 50, 35, 47,
        25, 16, 37, 27, 10, 54, 26, 58,
        62, 13, 18, 22, 21, 24, 12, 20,
        29, 38, 23, 32, 60, 34, 5, 11,
        45, 63, 40, 46, 52, 36, 17, 56
    ]

    P = [
        21, 8, 23, 6, 7, 15,
        22, 13, 19, 16, 25, 28,
        31, 32, 34, 36, 3, 39,
```

```

    29, 26, 24, 1, 43, 35,
    45, 12, 47, 17, 14, 11,
    27, 37, 41, 38, 40, 20,
    2, 0, 5, 4, 42, 18,
    44, 30, 46, 33, 9, 10
]

ROUND = 3
BLOCK_NUM = 8
MASK = (1 << (6 * BLOCK_NUM)) - 1

@classmethod
def _divide(cls, v: int) -> list[int]:
    l: list[int] = []
    for _ in range(cls.BLOCK_NUM):
        l.append(v & 0b111111)
        v >>= 6
    return l[::-1]

@staticmethod
def _combine(block: list[int]) -> int:
    res = 0
    for v in block:
        res <<= 6
        res |= v
    return res

@classmethod
def _sub(cls, block: list[int]) -> list[int]:
    return [cls.S[v] for v in block]

@classmethod
def _perm(cls, block: list[int]) -> list[int]:
    bits = ""
    for b in block:
        bits += f"{b:06b}"

    buf = ["_" for _ in range(6 * cls.BLOCK_NUM)]
    for i in range(6 * cls.BLOCK_NUM):
        buf[cls.P[i]] = bits[i]

    permd = "".join(buf)
    return [int(permd[i : i + 6], 2) for i in range(0, 6 * cls.BLOCK_NUM, 6)]

@staticmethod
def _xor(a: list[int], b: list[int]) -> list[int]:
    return [x ^ y for x, y in zip(a, b)]

def __init__(self, key: int):
    assert 0 <= key <= self.MASK

    keys = [key]
    for _ in range(self.ROUND):
        v = hashlib.sha256(str(keys[-1]).encode()).digest()
        v = int.from_bytes(v, "big") & self.MASK
        keys.append(v)

    self.subkeys = [self._divide(k) for k in keys]

def encrypt(self, inp: int) -> int:

```

```

        block = self._divide(inp)

        block = self._xor(block, self.subkeys[0])
        for r in range(self.ROUND):
            block = self._sub(block)
            block = self._perm(block)
            block = self._xor(block, self.subkeys[r + 1])

        return self._combine(block)

# TODO: Implement decryption
def decrypt(self, inp: int) -> int:
    raise NotImplementedError()

def handler(_signum, _frame):
    print("Time out!")
    exit(0)

def main():
    signal.signal(signal.SIGALRM, handler)
    signal.alarm(300)
    key = int.from_bytes(os.urandom(6), "big")

    cipher = SusCipher(key)

    while True:
        inp = input("> ")

        try:
            l = [int(v.strip()) for v in inp.split(",")]
        except ValueError:
            print("Wrong input!")
            exit(0)

        if len(l) > 0x100:
            print("Long input!")
            exit(0)

        if len(l) == 1 and l[0] == key:
            with open('flag', 'r') as f:
                print(f.read())

        print(", ".join(str(cipher.encrypt(v)) for v in l))

if __name__ == "__main__":
    main()

```

Let's take a look at the relevant parts

### **main**

While True, it asks for an input which is a string of numbers separated by , As long as we input 0x100 or 256 numbers at a time, we can get as many encryptions as we like

If we only enter a single number, and if that number happens to be the secret round key, we can get the flag

Sounds easy? lets take a look into the cipher

## The Cipher

The above construction is Substitution Permutation Network (SPN), which is essentially a repeated operation of substitution with a fixed predefined array here which is

```
S = [  
    43, 8, 57, 53, 48, 39, 15, 61,  
    7, 44, 33, 9, 19, 41, 3, 14,  
    42, 51, 6, 2, 49, 28, 55, 31,  
    0, 4, 30, 1, 59, 50, 35, 47,  
    25, 16, 37, 27, 10, 54, 26, 58,  
    62, 13, 18, 22, 21, 24, 12, 20,  
    29, 38, 23, 32, 60, 34, 5, 11,  
    45, 63, 40, 46, 52, 36, 17, 56  
]
```

I.e a number  $n$  is replaced with  $S[n]$

followed by a permutation of bits here defined by

```
P = [  
    21, 8, 23, 6, 7, 15,  
    22, 13, 19, 16, 25, 28,  
    31, 32, 34, 36, 3, 39,  
    29, 26, 24, 1, 43, 35,  
    45, 12, 47, 17, 14, 11,  
    27, 37, 41, 38, 40, 20,  
    2, 0, 5, 4, 42, 18,  
    44, 30, 46, 33, 9, 10  
]
```

Which means  $P[t]$ th bit of the output is actually the  $t$ th bit of input.e.g.  $P[0] = 21$  means the 21th bit of output is 0th bit of input as is

followed by a xor operation with a secret key

This process is repeated a fixed number of rounds times with a new secret key each round (which are called the round keys or subkeys)

The encrypt function hence looks like as below

```
def encrypt(self, inp: int) -> int:  
    block = self._divide(inp)  
  
    block = self._xor(block, self.subkeys[0])  
    for r in range(self.ROUND):  
        block = self._sub(block)
```

```

        block = self._perm(block)
        block = self._xor(block, self.subkeys[r + 1])

    return self._combine(block)

```

(`_divide` and `_combine` are just helper functions to make programmers life easier)

One might question, why are we `_dividing` a good enough input of 48 bits into 8 chunks of 6 bits each? Well, in an ideal world, we would like to have a substitution box of 48 bits, but that would eat up a whopping  $2^{48}$  number of entries (which we are somehow fooling with  $2^6$  entries here)

Hence the functions `_sub` acts as if it sees 8 different values and substitutes them and acts as if it just did 48 bits of substitution

```

@classmethod
def _sub(cls, block: list[int]) -> list[int]:
    return [cls.S[v] for v in block]

```

So does `_perm` pretend (because of our design) that it sees a big block of 48 bits which it permutes to a block of 48 bits, but what it does is to take 8 blocks of 6 bits each and create 8 blocks of 6 bits each if they were all connected

```

@classmethod
def _perm(cls, block: list[int]) -> list[int]:
    bits = ""
    for b in block:
        bits += f"{b:06b}"

    buf = ["_" for _ in range(6 * cls.BLOCK_NUM)]
    for i in range(6 * cls.BLOCK_NUM):
        buf[cls.P[i]] = bits[i]

    permd = "".join(buf)
    return [int(permd[i : i + 6], 2) for i in range(0, 6 * cls.BLOCK_NUM, 6)]

```

## Where do subkeys come from?

```

def __init__(self, key: int):
    assert 0 <= key <= self.MASK

    keys = [key]
    for _ in range(self.ROUND):
        v = hashlib.sha256(str(keys[-1]).encode()).digest()
        v = int.from_bytes(v, "big") & self.MASK
        keys.append(v)

    self.subkeys = [self._divide(k) for k in keys]

```

As you may have observed from the init function, subkeys are “derived” from a single 48-bit key in a way that we cant recover subkey  $i$  from the knowledge of any of the subkeys  $j \neq i$  (to make the challenge hard so that we will definitely need to recover subkey[0] which is the original key

## Vulnerability?

If you have seen some cipher constructions before, you may have observed, that the `ROUND = 3` is really very low and 6-bit sboxes are still not as robust as you may imagine them to be.

Another hint as provided by the author is [Differential Cryptanalysis](#), and since I am obsessed with SAT solvers, I will overlook the hint and cheeze it with z3

## Modelling

While the general methodology to solve a problem with a SAT solver is to write the output as a (symbolic) function of the inputs, and finding an input which leads to the observed output.

So what’s the symbolic input and output here?

For an input `inp` to the `SusCipher(key)` producing an encryption out We can write out as `symbolic_function(subkeys, inp)`

With subkeys acting as unknown `inp` which we aim for, we can easily get the desired outcome.

Taking heavy inspiration from the implementaion of the challenge cipher, we can similary create the z3 model of `suscipher`

```
class CrackSusCipher:
    ROUND = 3
    BLOCK_NUM = 8
    def _divide(self, v):
        l = []
        for _ in range(self.BLOCK_NUM):
            l.append(v&0b111111)
            v >>=6
        return l[::-1]

    def _combine(self, block):
        res = 0
        for v in block:
            res <<=6
            res |= v
        return res

    def _xor(self, a,b):
        return [x^y for x,y in zip(a,b)]
```

These functions look identical.

## Modelling substitution

First hurdle most of the people face modelling a SPN network or any other cipher is to model substitution.

But z3 is equipped with powerful theories of arrays (and functions)

Thus to model substitution, we can define a symbolic function  $S$ , which takes 6-bit inputs and generates 6-bit outputs

```
self.S = Function('S', BitVecSort(6), BitVecSort(6))
```

then  $\text{self.S}(i)$  would indeed be exactly what we desire

But wait, we just specified that  $S$  can be *any* function, not the exact substitution function we are provided with.

Worry not, we can specify this as a constraint to the solver

```
for i,v in enumerate(S): #original S as provided in the challenge
    self.solver.add(self.S(i)==v)
```

i.e we want  $S(0)$  to be nothing else than 43 and so on

And we treat keys as 6 bit unknowns, so there will be  $(\text{ROUND}+1)*8$  variables.

Overall our init function will look like

```
def __init__(self):
    self.S = Function('S', BitVecSort(6), BitVecSort(6))
    self.solver = Solver()
    for i,v in enumerate(S):
        self.solver.add(self.S(i)==v)
    self.keys = [[BitVec(f'k_{r}_{i}',6) for i in range(8) ] for r in
range(self.ROUND+1)]
```

hence  $\_sub$  function would be

```
def _sub(self, block):
    return [self.S(simplify(i)) for i in block]
```

Note that it could have been  $\text{self.S}(i)$  instead of  $\text{self.S}(\text{simplify}(i))$  which I used, just to simplify the expression (if possible) before substituting to hopefully speed things up

## Modelling permutation

Now what about the permutation? We can model it exactly how we would have calculated a permutation Take the  $i$ th bit, put it  $P[i]$ th place in the output, just the way to deal with BitVectors vary

```

def _perm(self, block):
    x = Concat(block)
    # treat the 8 6-bit vectors as a single 48 bit-vector
    output = [0]*48 # temporary placeholder for output
    for i,v in enumerate(P):
        # extract the ith bit from the MSB put it at the correct place
        output[v] = Extract(47-i, 47-i, x)
    # rechunk in 6 bit bitvectors
    return [Concat(output[i:i+6]) for i in range(0,48,6)]

```

## Modelling encryption

Finally after getting the required blocks to perform our symbolic encryption, we can model it

```

def enc(self, block):
    block = self._xor(block, self.keys[0])
    for r in range(self.ROUND):
        block = self._sub(block)
        block = self._perm(block)
        block = self._xor(block, self.keys[r+1])
    return block

```

Which you can see is almost like the original except we are not 'dividing and 'combining the 48-bits but rather assume that it operates on 8 6-bit values. And self.keys here are the symbolic unknowns.

## Checking if our model is correct

Now a CTF player will be anxious whether the efforts they put in to model the cipher were fruitful or did they mess up the model somewhere?

Worry not, we can always check our symbolic model by plugging in real values and comparing with the original cipher

We will use random values and keys just to check if they match (kinda funny that we have to informally verify a formal verifier XD)

```

print("verifying our modelling")
import random
for i in range(100):
    random_key = random.randint(0,2**48-1)
    sus = SusCipher(random_key)
    sus_model = crack()
    sus_model.solver.check() # to fill in the `S` as the original substitution
function
    sus_model.keys = [[BitVecVal(i,6) for i in row] for row in sus.subkeys] #
    BitVecVal as a symbolic constant value
    for j in range(10):
        inp = random.randint(0,2**48-1)
        real_out = sus.encrypt(inp)
        sym_out_chunks = sus_model.enc(sus_model._divide(inp))
        # evaluating the symbolic output as per the symbolic model

```

```

        sym_out = sus_model.solver.model().eval(Concat(sym_out_chunks))
        assert sym_out.as_long() == real_out
    print("success")

```

## Adding input output points

Taking care of the `divide` business, we will equate the 6-bit chunks of the output and our symbolic output for a given input

```

def add_sample(self, inp, oup):
    for a,b in zip(self.enc(self._divide(inp)), self._divide(oup)):
        self.solver.add(a==b)

```

## Getting the key

It's really simple, just check if there is any satisfying model which would make our constraints possible, and get the first subkey according to that model

```

def get(self):
    if self.solver.check()==sat:
        model = self.solver.model()
        k = [model.eval(i).as_long() for i in self.keys[0]]
        return self._combine(k)

```

## Putting our class together

```

S = [
    43, 8, 57, 53, 48, 39, 15, 61,
    7, 44, 33, 9, 19, 41, 3, 14,
    42, 51, 6, 2, 49, 28, 55, 31,
    0, 4, 30, 1, 59, 50, 35, 47,
    25, 16, 37, 27, 10, 54, 26, 58,
    62, 13, 18, 22, 21, 24, 12, 20,
    29, 38, 23, 32, 60, 34, 5, 11,
    45, 63, 40, 46, 52, 36, 17, 56
]

P = [
    21, 8, 23, 6, 7, 15,
    22, 13, 19, 16, 25, 28,
    31, 32, 34, 36, 3, 39,
    29, 26, 24, 1, 43, 35,
    45, 12, 47, 17, 14, 11,
    27, 37, 41, 38, 40, 20,
    2, 0, 5, 4, 42, 18,
    44, 30, 46, 33, 9, 10
]

class crack:
    ROUND = 3
    BLOCK_NUM = 8
    def __init__(self):
        self.S = Function('S', BitVecSort(6), BitVecSort(6))

```

```

        self.solver = Solver()
        for i,v in enumerate(S):
            self.solver.add(self.S(i)==v)
        self.keys = [[BitVec(f'k_{r}_{i}',6) for i in range(8) ] for r in
range(self.ROUND+1)]

    def _divide(self, v):
        l = []
        for _ in range(self.BLOCK_NUM):
            l.append(v&0b111111)
            v >>=6
        return l[::-1]

    def _combine(self, block):
        res = 0
        for v in block:
            res <=<=6
            res |= v
        return res

    def _xor(self, a,b):
        return [x^y for x,y in zip(a,b)]

    def _perm(self, block):
        x = Concat(block)
        output = [0]*48
        for i,v in enumerate(P):
            output[v] = Extract(47-i, 47-i, x)
        return [Concat(output[i:i+6]) for i in range(0,48,6)]

    def _sub(self, block):
        return [self.S(simplify(i)) for i in block]

    def enc(self, block):
        block = self._xor(block, self.keys[0])
        for r in range(self.ROUND):
            block = self._sub(block)
            block = self._perm(block)
            block = self._xor(block, self.keys[r+1])
        return block

    def add_sample(self, inp, oup):
        for a,b in zip(self.enc(self._divide(inp)), self._divide(oup)):
            self.solver.add(a==b)

    def get(self):
        if self.solver.check()==sat:
            model = self.solver.model()
            k = [model.eval(i).as_long() for i in self.keys[0]]
            return self._combine(k)

```

So how many input-output pairs do we need to figure out the key uniquely?

I guess atmost 256?

Let's try it out

```

c = crack()
for i in range(256):

```

```
input = random.randint(0,2**48-1)
output = server(input) #whatever
c.add_sample(input, output)

key = c.get()
```

Hmmm, something's not right, it seems to be stuck indefinitely.

We can get the intuition of difficulty of the solver to find key by reducing the number of constraints i.e the number of input output pairs.

By playing around, one quickly comes to the realisation that it wont workeven for 5 random samples and will time out ;200s

### Moment of inspiration

How about we address the difficulty of the solver (by addressing the difficulty of the problem being asked to solve)

When we take a random input-output pair, what we ask the solver for Substitution(key[i] ^ some\_random)

But if it were just 0 instead of some\_random, it would have to guess one less step.

So how about we make 7 out of 8 0 and only keep one key place active in substitution?.

This is really easy with input = (1<i) for (0≤i<48)

And most importantly, it works!

(To an amazement that it works in around a second with 48 samples as opposed to ~5000 seconds for 5 random samples!)

### Getting the flag

```
import pwn
HOST, PORT = "suscipher.chal.ctf.acsc.asia", 13579
REM = pwn.remote(HOST, PORT)

REM.sendline(",".join(str((1<i)) for i in range(48)))
response = list(map(int,REM.recvline()[2:].strip().split(b', ')))
c = crack()
for i,v in enumerate(response):
    c.add_sample(1<i,v)

key = c.get()
REM.sendline(str(key))
REM.interactive()
```

ACSC—There may be a better solution to solve this but I used diff analysis:("

As expected, the author knew there might be other interesting ways like this one ;)

Get the [Solve Script](#)

| Interactive controls are omitted from this PDF. See the canonical HTML version.